Ministerul Educației și Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Facultatea Calculatoare, Informatică și Microelectronică

Raport
la
Lucrarea de laborator Nr.7

Disciplina: Tehnici de programare

Tema: ” Trusa de unelte HTTP”

A efectuat : st. gr.SI – 261 Talchiv Corina

A verificat : asistent universitar Dumitrașcu Marius

Chișinău – 2026

SARCINA LUCRĂRII

Se construiește, pas cu pas, o trușă de funcții (httptools.py) care vorbesc cu un server web: trimit parametri în adresă, schimbă anteturi, trimit date (JSON și formular), interpretează codurile de stare, urmăresc redirecționările, păstrează cookie-uri și produc un raport complet despre o pagină. Serverul de test este httpbin.org, care răspunde cu o descriere a ceea ce i s-a trimis. Bonus: autentificare HTTP Basic și mutarea lui main() într-un fișier separat, main.py.

Constrângări din enunț: numele și parametrii funcțiilor nu se schimbă; adresa serverului se ia din constanta BASE; funcțiile din Partea 7 nu au voie să se oprească cu eroare.

Scopul și obiectivele

Scop: înțelegerea felului în care un client (script Python) comunică cu un server prin HTTP și folosirea corectă a bibliotecii requests.

Obiective:

1. trimiterea parametrilor în URL cu params=;
2. setarea anteturilor cu headers= și observarea User-Agent-ului implicit;
3. trimiterea datelor cu json= și cu data=, și diferența dintre ele;
4. clasificarea codurilor de stare (2xx, 4xx, 5xx) și raise_for_status() cu try/except;
5. urmărirea redirecționărilor (r.history, r.url) și oprirea lor cu allow_redirects=False;
6. păstrarea cookie-urilor cu requests.Session();
7. gestionarea erorilor de rețea (timeout, RequestException) și scrierea unui raport robust;
8. lucrul cu dicționare, liste, sorted(..., key=...) și module (import).

Implementarea și ce se afișează la ecran

Constante:

```
import requests

BASE = "https://httpbin.org"
PATHS = ["/get", "/redirect/2", "/delay/1", "/status/404", "/delay/5"]
TIMEOUT = 10
```

BASE este singurul loc în care apare adresa serverului; toate funcțiile o construiesc cu f"{BASE}/get". PATHS este o **listă** de căi pe care le va parcurge raportul final. TIMEOUT este o constantă pentru cererile simple: fără timeout, o cerere se poate bloca la nesfârșit.

Partea 1: parametri în URL

```
def search(params):
    r = requests.get(f"{BASE}/get", params=params, timeout=TIMEOUT)
    return r.json().get("args")

def search_url(params):
    r = requests.get(f"{BASE}/get", params=params, timeout=TIMEOUT)
    return r.url
```

params= primește un dicționar și construiește singur partea ?cheie=valoare&... din adresă, codificând caracterele speciale. Serverul răspunde cu un JSON care are cheia args, adică exact parametrii primiți; .get("args") extrage această cheie.

Partea 2: anteturi

```
def fetch_with_agent(agent):
    r = requests.get(f"{BASE}/headers", headers={"User-Agent": agent}, timeout=TIMEOUT)
    return r.json()["headers"].get("User-Agent")

def default_agent():
    r = requests.get(f"{BASE}/headers", timeout=TIMEOUT)
    return r.json()["headers"].get("User-Agent")
```

`headers=` este un dicționar de anteturi. Serverul răspunde cu anteturile pe care le-a văzut, într-un dicționar `headers`.

Partea 3: trimiterea datelor

```
def post_json(data):
    r = requests.post(f"{BASE}/post", json=data, timeout=TIMEOUT)
    return r.json().get("json")

def post_form(data):
    r = requests.post(f"{BASE}/post", data=data, timeout=TIMEOUT)
    return r.json().get("form")
```

Corecție față de varianta inițială: funcțiile foloseau print(...) și se terminau cu pass, deci întorceau None. Enunțul cere return, iar un test automat ar fi primit None. Acum returnează valoarea cerută.

Diferența între cele două: json= serializează dicționarul ca JSON (păstrează tipurile: numere, liste), iar data= îl trimite ca câmpuri de formular HTML (totul devine șir de caractere). Serverul pune răspunsul într-o cheie care poartă numele formatului (json sau form); cu argumentul greșit, cheia potrivită rămâne goală.

Partea 4: coduri de stare

```
def describe_status(code):
    r = requests.get(f"{BASE}/status/{code}", timeout=TIMEOUT)
    if 200 <= r.status_code < 300:
        return "succes"
    if 400 <= r.status_code < 500:
        return "eroare client"
    if 500 <= r.status_code < 600:
        return "eroare server"
    return "altceva"

def is_ok(code):
    try:
        r = requests.get(f"{BASE}/status/{code}", timeout=TIMEOUT)
        r.raise_for_status()
        return True
    except requests.RequestException:   # include HTTPError (4xx/5xx)
        return False
```

Implicit, `requests` nu dă eroare la 404 sau 500; răspunsul vine normal, iar `describe_status` doar îl clasifică pe intervale. `is_ok` folosește `raise_for_status()`, care ridică `HTTPError` pentru 4xx/5xx; excepția e prinsă cu `except requests.RequestException` (clasa-părinte, deci acoperă și erorile de rețea), iar funcția returnează `False` în loc să se oprească. Ramura finală `"altceva"` acoperă codurile 1xx/3xx, pe care enunțul nu le definește.

Partea 5: redirecționări

```
def count_redirects(n):
    r = requests.get(f"{BASE}/redirect/{n}", timeout=TIMEOUT)
    return len(r.history)

def final_url(n):
    r = requests.get(f"{BASE}/redirect/{n}", timeout=TIMEOUT)
    return r.url

def first_hop(n):
    r = requests.get(f"{BASE}/redirect/{n}", allow_redirects=False, timeout=TIMEOUT)
    return r.headers.get("Location")
```

`requests` urmează redirecționările automat și și le ține în `r.history` (o listă cu un răspuns pentru fiecare pas), deci `len(r.history)` dă numărul lor. `r.url` este adresa finală. Cu `allow_redirects=False` cererea se oprește la primul răspuns (cod 302), iar antetul `Location` spune unde voia serverul să meargă clientul. S-a folosit `.get("Location")` în loc de `["Location"]`: dacă antetul lipsește, se returnează `None` în loc de `KeyError`.

Partea 6: sesiuni și cookie-uri

```
def cookie_demo(name, value):
    with requests.Session() as s:
        s.get(f"{BASE}/cookies/set", params={name: value}, timeout=TIMEOUT)
        r = s.get(f"{BASE}/cookies", timeout=TIMEOUT)
```

```
    return r.json().get("cookies")

def no_session_demo(name, value):
    requests.get(f"{BASE}/cookies/set", params={name: value}, timeout=TIMEOUT)
    r = requests.get(f"{BASE}/cookies", timeout=TIMEOUT)
    return r.json().get("cookies")
```

`/cookies/set` răspunde cu `Set-Cookie` și redirecționează la `/cookies`. În `cookie_demo`, ambele cereri trec prin același obiect `Session`, care păstrează cookie-ul și îl retrimite. În `no_session_demo`, fiecare `requests.get` pornește de la zero, iar a doua cerere „uită" cookie-ul. Blocul `with` închide sesiunea automat. Pentru `params={name: value}` am folosit dicționar în loc de a lipi `?name=value` în adresă, ca valorile speciale să fie codificate corect.

Partea 7: raportul final

```
def report(path):
    try:
        r = requests.get(BASE + path, timeout=3)
        return {
            "status": r.status_code,
            "final_url": r.url,
            "redirects": len(r.history),
            "seconds": round(r.elapsed.total_seconds(), 2),
            "content_type": r.headers.get("Content-Type"),
        }
    except requests.RequestException as e:
        return {"status": None, "error": type(e).__name__}
```

Funcția returnează un **dicționar** cu cheile cerute. `timeout=3` limitează așteptarea la 3 secunde. `r.elapsed` este un `timedelta`; `.total_seconds()` îl transformă în număr, rotunjit la două zecimale. La orice eroare de rețea (`Timeout`, `ConnectionError`) se prinde `requests.RequestException` și se returnează `{"status": None, "error": "<nume>"}`; mesajul scurt este numele clasei excepției, ca să nu apară texte lungi cu adrese interne. `.get("Content-Type")` evită `KeyError` dacă antetul lipsește.

7b. Bucla din `main()`:

```
def main():

    rezultate = []

    for path in PATHS:

        d = report(path)

        d["path"] = path  # adaug adresa în dicționar

        rezultate.append(d)



    reusite = [d for d in rezultate if d["status"] == 200]
```

```
    for d in sorted(reusite, key=lambda d: d["seconds"], reverse=True):
        print(f"{d['path']} -> {d['status']}, {d['seconds']} secunde")


    print(f"Reusite: {len(reusite)} | Esuate: {len(rezultate) - len(reusite)}")
```

Se parcurge lista `PATHS`, iar fiecare dicționar primește cheia `path`, ca adresa să poată fi afișată. List comprehension-ul `reusite` păstrează doar paginile cu status exact `200` (deci `/status/404` și `/delay/5` cad, iar pentru `/delay/5` `d["status"]` este `None`, comparat fără eroare). `sorted(..., key=lambda d: d["seconds"], reverse=True)` ordonează după timp, cea mai lentă prima. Numărul eșecurilor este total minus reușite.

```
(.venv) PS D:\Securitatea_Informationala_UTM\Tehnici_de_Programare\tehnici-prog-talchiv_C>
ramare\tehnici-prog-talchiv_C\L7_TP_Corina_T\main.py"
/delay/1 -> 200, 1.81 secunde
/get -> 200, 1.05 secunde
/redirect/2 -> 200, 0.14 secunde
Reusite: 3 | Esuate: 2
```

Figură 1

Partea 8 (bonus): autentificare și module

8a. check_login:

```
def check_login(user, password):
    try:
        r = requests.get(f"{BASE}/basic-auth/admin/secret",
                         auth=(user.encode("utf-8"), password.encode("utf-8")),
                         timeout=TIMEOUT)
        return r.status_code == 200
    except (requests.RequestException, UnicodeError):
        return False
```

`auth=(user, parolă)` trimite autentificarea HTTP Basic; serverul răspunde 200 dacă datele sunt corecte și 401 dacă nu. Funcția returnează `True` doar la 200. **Capcană găsită la testare:** exemplul din enunț, `check_login("admin", "greșit")`, conține litera `ș`, iar `requests` codifică șirurile din `auth` în `latin-1`, care nu are acest caracter. Rezultatul era `UnicodeEncodeError` (nu este `RequestException`), deci programul se oprea cu eroare. Soluția: parola și utilizatorul se convertesc în octeți UTF-8 (`.encode("utf-8")`), iar `UnicodeError` este prins și el.

8b. main.py:

```
from httptools import PATHS, report

def main():
    rezultate = []
    for path in PATHS:
        d = report(path)
        d["path"] = path
        rezultate.append(d)

    reusite = [d for d in rezultate if d["status"] == 200]
    for d in sorted(reusite, key=lambda d: d["seconds"], reverse=True):
        print(f"{d['path']} -> {d['status']}, {d['seconds']} secunde")

    print(f"Reusite: {len(reusite)} | Esuate: {len(rezultate) - len(reusite)}")

if __name__ == "__main__":
    main()
```

`main.py` importă din `httptools` ce î i trebuie (`PATHS`, `report`) și rulează programul complet cu `python main.py`. Fișierul `httptools.py` rămâne complet și funcțional, cu `main()` păstrat în el, cum cere enunțul. Blocul `if __name__ == "__main__":` face ca `main()` să ruleze doar la pornirea directă, nu la `import`.

```
=== Partea 1 ===
{'page': '2', 'q': 'python'}
{'q': 'hello world & co'}
https://httpbin.org/get?q=python&page=2
=== Partea 2 ===
MyBot/1.0
python-requests/2.34.2
=== Partea 3 ===
{'age': 19, 'name': 'Ana', 'tags': ['a', 'b']}
{'city': 'Bălți', 'user': 'ana'}
=== Partea 4 ===
succes eroare client eroare server
True False
=== Partea 5 ===
3
https://httpbin.org/get
/relative-redirect/2
=== Partea 6 ===
{'theme': 'dark'}
{}
=== Partea 7 ===
/get {'status': 200, 'final_url': 'https://httpbin.org/get', 'redirects': 0, 'seconds': 0.8, 'content_type': 'application/json'}
/redirect/2 {'status': 200, 'final_url': 'https://httpbin.org/get', 'redirects': 2, 'seconds': 0.14, 'content_type': 'application/json'}
/delay/1 {'status': 200, 'final_url': 'https://httpbin.org/delay/1', 'redirects': 0, 'seconds': 2.28, 'content_type': 'application/json'}
/status/404 {'status': 404, 'final_url': 'https://httpbin.org/status/404', 'redirects': 0, 'seconds': 1.23, 'content_type': 'text/html; charset=utf-8'}
/delay/5 {'status': None, 'error': 'ReadTimeout'}
=== Programul complet ===
/delay/1 -> 200, 1.82 secunde
/get -> 200, 0.98 secunde
/redirect/2 -> 200, 0.41 secunde
Reusite: 3 | Esuate: 2
=== Bonus ===
True
False
```

Figură 2 – Rezultatele executării

CONCLUZII

În lucrare am implementat și testat trusa HTTP completă: parametri în URL, anteturi, trimitere JSON și formular, clasificarea codurilor de stare, redirecționări, sesiuni cu cookie-uri, raportul final robust, autentificarea Basic și separarea în module. Am văzut în practică că:

- params=, json= și data= lasă biblioteca să codifice datele, în loc să le lipim de mână;
- requests nu dă eroare la 4xx/5xx din oficiu: eroarea apare doar cu raise_for_status(), iar o cerere fără răspuns (timeout, conexiune) se prinde cu RequestException;
- r.history păstrează traseul redirecționărilor, iar allow_redirects=False permite să-l oprim la primul pas;
- fiecare requests.get simplu uită cookie-urile, pe când Session le ține minte;
- un raport bun separă trei situații: pagina a răspuns cu succes, serverul a răspuns cu eroare (404) și nu am primit răspuns deloc (timeout);
- dicționarele, listele, comprehensions și sorted(..., key=...) fac prelucrarea rezultatelor scurtă și clară;
- testarea cu date neobișnuite (diacritice în parolă) poate scoate la iveală erori pe care testele simple nu le prind.